

1. In the notebook, the audio waveform is transformed into time-frequency features before being passed to the keyword spotting model. What is the purpose of converting raw audio into spectrogram/MFCC-style features instead of directly using the raw time-domain waveform?

The raw time-domain isn't helpful for our case as this project is time independent from what happens in real life. For example, the latency matters much more than having some type of prediction on when the user would say a keyword. As we are working with constrained resources, switching to the frequency domain and putting thresholds on those is much easier and faster than directly using the low raw time domain waveforms.

2. During full integer quantization, the notebook uses a representative dataset. What is the purpose of the representative dataset, and why is it important for creating an INT8 TensorFlow Lite model?

There were two different types of model compositions on the notebook which are int8 and a floating-point representation of the model. So, the representative dataset can be helpful to alternate between those datatypes or different ones based on the devices we are trying to implement. But int8 TensorFlow lite model is very helpful to consuming very little working memory on the Arduino nano board where Ram is extremely limited.

3. What does `audio_processor.get_data()` do? Refer to `input_data.py`.

It captures all the data from background frequencies for silent mode, sample testing without the raw audio data. So it has a data pipeline of audio datasets which returns a batch of preprocessed featured vectors.

4. In the final model export step, the TensorFlow Lite model is converted into a C/C++ byte array. What is `g_model` and `g_model_len`, and why do they need to be copied correctly into the Arduino source file?

We need to copy is down exactly to have the same length as the array of all the binaries as devices like Arduino or other embedded chips need to have executable file rather than OS or file system. And we are getting the g model len to make sure we are selecting all the model information in hex values based on the size we gave on the Arduino idea.

5. Please include a screenshot showing the correct identification of at least one keyword, either **“YES”** or **“NO”**, as displayed in the Arduino IDE Serial Monitor after

deploying your KWS model on the Arduino Nano 33 BLE. The screenshot should be included in your submitted PDF.

```
12:47:48.261 -> Heard silence (234) @1056ms
12:47:48.336 -> Heard silence (222) @1136ms
12:47:48.449 -> Heard silence (218) @1232ms
12:47:48.522 -> Heard silence (211) @1328ms
12:47:48.630 -> Heard silence (205) @1424ms
12:47:50.588 -> Heard unknown (207) @3360ms
12:48:07.610 -> Heard unknown (206) @3808ms
12:48:12.197 -> Heard unknown (201) @5376ms
12:48:19.379 -> Heard unknown (207) @12544ms
12:48:23.264 -> Heard yes (207) @16416ms
12:48:24.169 -> Heard unknown (211) @17344ms
12:48:26.470 -> Heard yes (205) @19648ms
12:48:28.050 -> Heard unknown (204) @21200ms
12:48:31.072 -> Heard unknown (209) @24240ms
12:48:34.968 -> Heard unknown (206) @28112ms
12:48:39.076 -> Heard no (202) @32256ms
12:48:40.660 -> Heard unknown (207) @33808ms
12:48:43.325 -> Heard unknown (201) @36480ms
```

As result shown above with detecting silence very accurately and yes too but it was mixing with NO and unknown command for bit as assuming the data collection from audio wasn't perfect or needed more samples of NO keyword.